DOKUMENTASI PBS

PoolTick

Nama	: Nazmi Mustafa
NPM	: 23312213
JobDes	: CMS, USER
Nama Kelompok	: Teluk On Top

Daftar Isi

Daftar Isi 2
CMS 3
1. Menginisialisasi aplikasi NestJS (main.ts). 3
2. Sebagai "pengikat" atau orchestrator dari semua komponen (app.module.ts). 3
3. Menangani request HTTP yang masuk ke CMS (app.controller.ts) 4
4. Konfigurasi Alamat API 4
5. Bagian Mengambil Data 4
6. Bagian Mengirim/Menyimpan Data 5
7. Bagian Proses Pembelian 6
8. Bagian Penghapusan Data 6

CMS

1. Menginisialisasi aplikasi NestJS (main.ts).

```
1  import { NestFactory } from '@nestjs/core';
2  import { AppModule } from './app.module';
3  import { join } from 'path';
4  import * as express from 'express';
5
6  async function bootstrap() {
7    const app = await NestFactory.create(AppModule);
8
9    app.use(express.static(join(__dirname, '..', 'src', 'views')));
10
11   await app.listen(3001);
12 }
13 bootstrap();
```

Bagian baris nomor 11 “await app.listen(3001)” itu saya bedakan dari localhost dari backend. Jadi untuk memanggil CMS yaitu dengan localhost:3001.

Dan untuk bagian baris nomor 9 “express.static(join(__dirname, '..', 'src', 'views')))” ini ada bagian untuk memanggil tampilan interface dari CMS.

2. Sebagai "pengikat" atau orchestrator dari semua komponen (app.module.ts).

```
1  import { Module } from '@nestjs/common';
2  import { AppController } from './app.controller';
3  import { AppService } from './app.service';
4
5  @Module({
6    imports: [],
7    controllers: [AppController],
8    providers: [AppService],
9  })
10 export class AppModule {}
11
```

File ini digunakan sebagai Root Module atau module utama dalam aplikasi CMS Pooltick. Fungsinya sebagai pusat registrasi komponen utama aplikasi agar dapat dikelolah NestJs Container.

3. Menangani request HTTP yang masuk ke CMS (app.controller.ts)

```
1  import { Controller, Get } from '@nestjs/common';
2  import axios from 'axios';
3
4  @Controller()
5  export class AppController {
6
7    @Get('tickets')
8    async getTickets() {
9      const res = await axios.get('http://localhost:3000/tickets');
10     return res.data;
11   }
12 }
```

Ini adalah file app controller untuk di bagian baris nomor 7 "@Get('tickets')" ini adalah endpoint untuk CMS menggambil data tiket.

Dan untuk bagian baris nomor 9 "*await* axios.get('http://localhost:3000/tickets')" bahwasan ini CMS bertindak sebagai client yang melakukan request ke API/Backend.

4. Konfigurasi Alamat API

```
1  const API_TICKETS = 'http://localhost:3000/tickets';
2  const API_TRANSACTIONS = 'http://localhost:3000/transactions';
```

Ini adalah code dari folder scr-views-index.html. Code ini variabel dasar yang menentukan kemana CMS harus meminta data. Semua CRUD akan lari ke URL ini di localhost:3000 yaitu bagian API/Backend.

5. Bagian Mengambil Data

```
1  const [tickets, transactions] = await Promise.all([
2      fetch(API_TICKETS).then(r => r.json()),
3      fetch(API_TRANSACTIONS).then(r => r.json())
4    ]);
```

Code ini menggunakan fetch yang berguna sebagai menarik data mentah dari API. Karena CMS tidak memiliki database sendiri dan sangat bergantung data yang dikirim dari API.Cara

kerjanya:

- **Asynchronous Request (await):** CMS menggunakan perintah await yang artinya sistem akan menunggu sampai data dari API benar-benar terkirim sebelum mencoba menampilkannya. Ini mencegah aplikasi error karena mencoba menampilkan data yang belum ada.
- **Parallel Fetching (Promise.all):** Ini teknik tingkat tinggi. CMS mengambil data Tiket dan data Transaksi secara bersamaan (paralel). Ini membuat aplikasi terasa jauh lebih cepat daripada mengambil datanya satu per satu.
- **Data Synchronization:** Setelah data mentah (JSON) diterima, CMS langsung memprosesnya ke fungsi renderTickets dan renderTransactions untuk mengubah kode-kode komputer menjadi tabel yang bisa dibaca manusia.

6. Bagian Mengirim/Menyimpan Data

```
const method = id ? 'PATCH' : 'POST';
  const url = id ? `${API_TICKETS}/${id}` : API_TICKETS;

  await fetch(url, {
    method, headers: {'Content-Type':'application/json'},
    body: JSON.stringify({ name, price })
  });
  resetForm();
  loadData();
}
```

Code di baris ini menentukan apakah CMS sendang dalam menambahkan tiket baru yaitu dengan POST atau mengedit tiket lama yaiyu PATCH. Data dikirim dalam format JSON ke API/Backend.

Cara kerja:

- **Dynamic Method Selection:** Kode kamu sangat efisien karena menggunakan satu fungsi untuk dua kegunaan.
    - Jika ada ID, CMS menggunakan metode PATCH (Update).
    - Jika ID kosong, CMS menggunakan metode POST (Tambah Baru).
- **Data Encapsulation:** Data seperti Nama Tiket dan Harga dibungkus ke dalam format JSON.stringify. Ini adalah standar industri untuk pengiriman data antar aplikasi (dari CMS ke API).

- **Header Configuration:** Penggunaan 'Content-Type': 'application/json' memberi tahu Backend: "Woi Backend, saya kirim paket berupa data JSON, tolong dibongkar sesuai format ya!".

7. Bagian Proses Pembelian

```
for (let i = 0; i < qty; i++) {
      await fetch(API_TRANSACTIONS, {
        method: 'POST', headers: {'Content-Type':'application/json'},
        body: JSON.stringify({
          ticketId: t.id, name: buyer, price: t.price, createdAt: new Date().toISOString()
        })
      });
    }
```

Code ini adalah inti dari sisi USER/PELANGGAN. Setiap tombol “Checkout” ditekan, CMS akan mengirimkan data transaksi satu per satu ke database API/Backend.

Cara kerja:

- **Looping Transaksi:** Kode menggunakan for (let i = 0; i < qty; i++). Ini berarti jika user membeli 3 tiket sekaligus, CMS akan melakukan 3 kali pengiriman data (request) ke API secara otomatis.
- **Payload (Data yang dikirim):** CMS membungkus data berupa ticketId (ID tiket), name (Nama pembeli), price (Harga saat itu), dan createdAt (Waktu transaksi).
- **Metode HTTP POST:** CMS menggunakan metode POST ke endpoint API_TRANSACTIONS. Ini memberi instruksi ke Backend: "Tolong buatkan satu catatan transaksi baru dengan data ini."

8. Bagian Penghapusan Data

```
1  await fetch(`${API_TICKETS}/${id}`, { method: 'DELETE' });
2  fetch(`${API_TRANSACTIONS}/${id}`, {method:'DELETE'}).then(loadData);
```

Untuk bagian code ini instruksi langsung dari backend untuk menghapus data tertentu berdasarkan ID uniknya.

Cara kerja:

- **Metode HTTP DELETE:** CMS mengirimkan request dengan metode DELETE yang diikuti oleh ID spesifik di akhir URL-nya (contoh:

localhost:3000/tickets/5). Ini memastikan hanya data dengan ID tersebut yang terhapus.

- **Validasi Integritas:** Perhatikan kode if (trx.some(t => t.ticketId == id)). Ini adalah fitur cerdas di CMS kamu. Sebelum menghapus tiket, CMS mengecek ke API: "Apakah tiket ini pernah dipakai transaksi?". Jika iya, penghapusan ditolak agar laporan keuangan tidak error (data tiket hilang tapi riwayat transaksi masih ada).
- **Refresh Otomatis:** Setelah perintah DELETE berhasil direspon oleh API, CMS langsung menjalankan loadData(). Ini sinkronisasi agar tampilan di layar langsung berubah tanpa user harus menekan tombol refresh browser (F5).